Interview AI: AI - Powered Interview Question Generator

Project Description:

InterviewAI is a state-of-the-art recruiter empowerment platform that harnesses Generative AI to deliver highly customized, role-specific, and context-aware interview questions instantly. Built with a sleek, premium single-page web interface and backed by a robust Flask microservice, InterviewAI automates the cognitive burden of generating balanced technical, behavioral, and situational questions tailored precisely to experience levels and unique organizational contexts. It integrates seamlessly with the Google Gemini API to guarantee fast, reliable response generation, coupled with a thorough evaluation framework — complete with green flags, red flags, and scoring scales — to help recruiters conduct structured, objective, and unbiased candidate assessments.

By leveraging environment-based key management for secure enterprise operation, persistent local JSON history tracking for easy session retrieval, and automated document compilation using python-docx, InterviewAI establishes a secure, standard-compliant tool. It eliminates repetitive administrative prep work, allowing HR professionals and technical managers to focus entirely on candidate engagement.

Scenarios:

Scenario 1: A technical lead is preparing to interview a candidate for a 'Senior React Developer' role. By selecting an Advanced experience level and mixed interview type, the lead generates ten customized questions, complete with follow-up prompts and evaluation tips. The system also supplies green flags to watch for and red flags to avoid, ensuring the evaluation is highly objective.

Scenario 2: An HR manager needs to conduct a soft-skills screening for a 'Product Manager' role. By choosing the 'Behavioral' interview type, the manager generates situational and behavioral questions that target agile methodologies, roadmapping, and conflict resolution, without needing deep technical expertise.

Scenario 3: A hiring team wants to conduct a quick screening. They generate 5 beginner-level questions. After the session, they export the generated questions to a clean Microsoft Word (.docx) document, formatted professionally and ready to be printed or shared across the department.

Scenario 4: A recruiter wants to review an interview set from last week. They navigate to the sidebar history or the landing page dashboard, click on the saved 'Senior React Developer' session, and instantly reload the entire questions dashboard without making a new API request, saving time and API resource quota.

Technical Architecture

The technical architecture of InterviewAI follows a decoupled, asynchronous Single-Page Application (SPA) design, separating client-side presentation from backend business logic. This separation ensures extreme responsiveness and easy deployment. The diagram below illustrates the complete component flow across all three layers of the system:

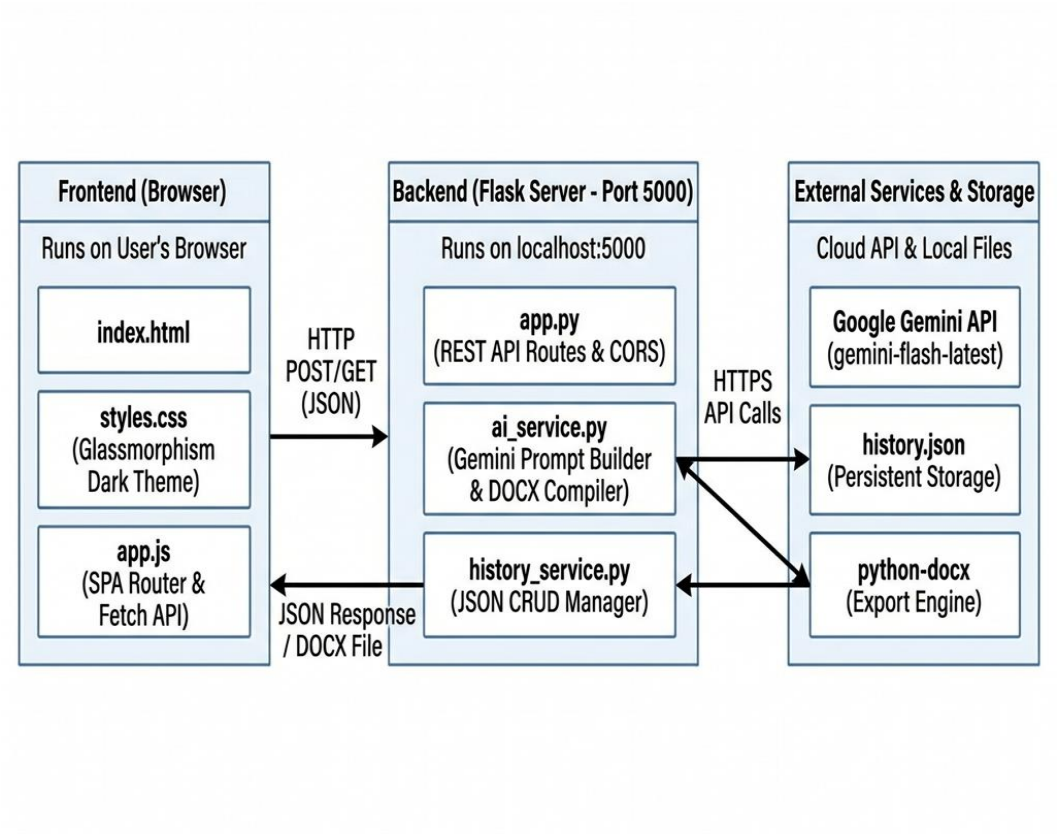


Figure: InterviewAI Technical Architecture — Three-Layer Component Flow Diagram

The Frontend Layer runs entirely in the user's browser using standard HTML5, CSS3, and ES6 JavaScript. It communicates with the Backend Layer via asynchronous HTTP POST and GET requests carrying JSON payloads. The Backend Layer, built on Flask micro-framework, hosts RESTful API endpoints for question generation, history management, and document export. It connects securely to the Google Gemini API over HTTPS for AI-powered question generation, and manages local persistent storage through a structured JSON file.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Google Gemini API Familiarity:** Google Gemini API
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Ngrok for Public Deployment:** Ngrok Documentation

Project Workflow

The development of the InterviewAI platform was executed systematically through a structured, multi-phase workflow. Each phase (Activity) was designed to build upon the accomplishments of the previous phase, ensuring strict standard compliance and code quality throughout the lifecycle. Below is a detailed breakdown of all activities performed during the project development:

Activity 1: Model Selection and Architecture

This activity focused on establishing the foundational infrastructure for the entire project. It involved generating and securely configuring a Google Gemini API key within the project environment, researching and evaluating multiple Generative AI models to identify the optimal choice for low-latency web generation, defining the complete logical architecture detailing data flow between the frontend and backend layers, and establishing the development workspace by installing Flask, dotenv, and all necessary generative AI packages inside an isolated Python virtual environment.

- **Activity 1.1:** Generate a secure Google Gemini API key and register in the project environment.
- **Activity 1.2:** Research and select the appropriate Generative AI model for fast response latency.
- **Activity 1.3:** Define the logical architecture, detailing data flow between the frontend and backend.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Gemini API.

Activity 2: Core Functionalities Development

This activity centered on building the core intelligence and data management layer of the application. It involved developing sophisticated AI generation prompts that instruct Google Gemini to produce technical, behavioral, and situational interview questions with proper difficulty distribution. Additionally, a persistent local history storage system was implemented to log, read, and delete interview sessions using structured JSON, and a python-docx file compilation service was built to export generated questions as professionally formatted Word documents.

- **Activity 2.1:** Develop core AI generation prompts for technical, behavioral, and situational segments.
- **Activity 2.2:** Implement persistent local history storage to log, read, and delete interview sessions.
- **Activity 2.3:** Implement python-docx file compilation to export generated questions professionally.

Activity 3: App.py Backend Development

This activity focused on implementing the Flask micro-framework backend that serves as the central nervous system of the application. It involved writing the main Flask initialization with CORS setup and static file serving, implementing the /api/generate endpoint with comprehensive data validation and error handling, building RESTful history CRUD handlers for listing, reading, and deleting sessions, and creating the /api/export endpoint that compiles question data into downloadable Word documents.

- **Activity 3.1:** Write main Flask initialization, setup CORS, and route standard index path.
- **Activity 3.2:** Implement /api/generate handler, including data validation and try-except safety.
- **Activity 3.3:** Implement history CRUD handlers for listing, reading individual, and deleting sessions.
- **Activity 3.4:** Implement /api/export handler for Word document compilation and download streaming.

Activity 4: Frontend Development

This activity addressed the entire user interface layer. It involved designing a modern responsive layout using CSS with a customizable dark-theme glassmorphism aesthetic, implementing a Single-Page Application (SPA) router in vanilla JavaScript with async form submission using the Fetch API, and refactoring the homepage flow by centering hero content, adding an 'Explore More' reveal interaction, and creating a premium landing experience with animated glow orbs.

- **Activity 4.1:** Design modern responsive layout using CSS containing customizable dark-theme glassmorphism.
- **Activity 4.2:** Implement SPA single-page view router in vanilla JS and async form submission using Fetch.
- **Activity 4.3:** Refactor homepage flow by centering hero content, removing visual cards, and adding Explore More.

Activity 5: Deployment & Verification

This final activity focused on verifying, validating, and deploying the completed system. It involved preparing the server environment and verifying all local network calls via Chrome DevTools, confirming CORS headers are active and API responses are correct, and exposing the local port publicly using Ngrok tunnels to support cross-device mobile testing under real network conditions.

- **Activity 5.1:** Prepare server environment and verify local network calls via Chrome DevTools.
- **Activity 5.2:** Expose local port publicly using Vercal to support mobile testing.

Milestone 1: Model Selection and Architecture

The primary focus of Milestone 1 is to establish a secure project environment, evaluate and select the best artificial intelligence model, and set up the structural foundation of the workspace. This ensures the application is built on a compliant, secure, and highly scalable basis.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Google Generative AI servers, we must obtain a valid API key and configure it securely within the local project workspace. Below are the precise steps followed:

- **Step 1 — Create a Google AI Studio Account:** Visit the Google AI Studio page (https://aistudio.google.com/) and register using a developer or enterprise account.
- **Step 2 — Generate the API Key:** Navigate to the 'Get API Key' section, select 'Create API Key in New Project', and copy the generated secure token string immediately.
- **Step 3 — Local Security Setup:** In the project root folder, verify that a file named .env is present. Save the key using the exact variable name expected by the service wrapper.

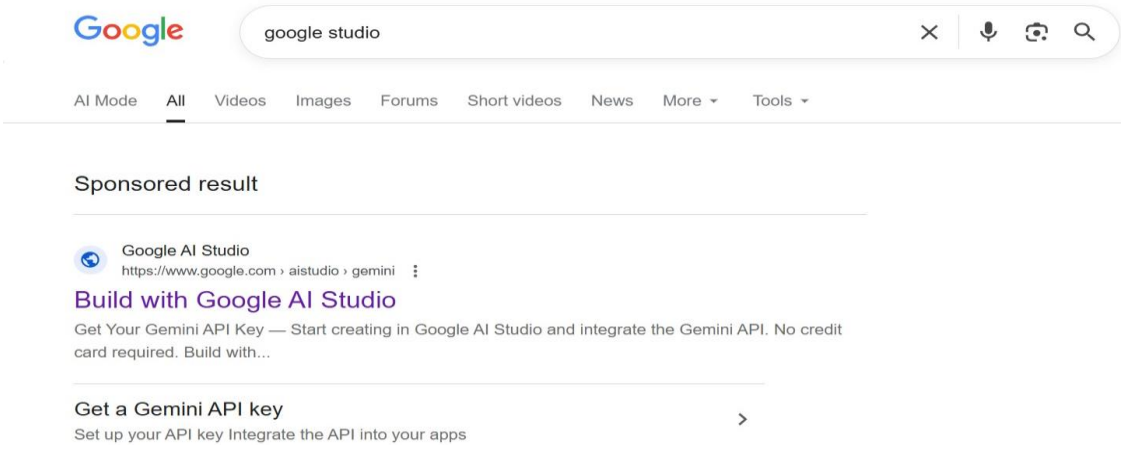


Figure: Google AI Studio — Searching for Google Studio to access the API Key portal.

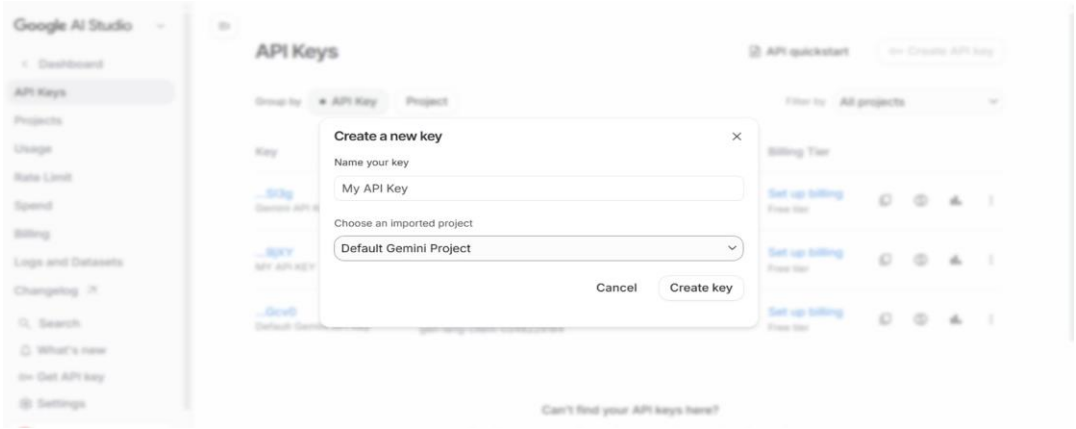


Figure: Google AI Studio — Creating a new API Key in the 'Get API Key' dashboard.

Here is the structure used inside the project's configuration file:

```
GEMINI_API_KEY=your_api_key_here
PORT=5000
```

To guarantee that this API key is never leaked or committed to source control repositories, the .env file is explicitly added to the project's .gitignore file, enforcing responsible developer best practices.

Activity 1.2: Research and Select the Generative AI Model

Selecting the optimal large language model requires a thorough balance between response latency, context window capacity, API request quota limits, and output format reliability. We evaluated several options:

- **Gemini 1.5 Pro:** Offers extremely deep reasoning and a massive 2-million token context window, but has a higher latency and lower request-per-minute limits on the free tier, making it suboptimal for fast web generation.
- **Gemini 2.5 Flash:** Specifically engineered for high speed, low latency, and highly efficient processing. It features a robust JSON formatting mode and boasts a generous free-tier rate limit, making it the perfect candidate for instant generation.

Consequently, we chose the 'gemini-flash-latest' model. In addition, we configured the generator to operate at a temperature of 0.7, encouraging high creative variety in behavioral/situational questions while maintaining strict professional standards.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application — entering Job information, selecting an experience Level , choosing Number Of Questions and Type of Questions(Behaviour,technical,mixed.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Gemini API.
- **Describe AI Integration Points:** Define how the backend constructs prompts, calls the Gemini API, parses the JSON response via extract_json(), and returns structured results to the frontend.

Activity 1.4: Project Structure

The workspace structure is organized logically with a clear separation between backend services and frontend presentation layers as shown below:

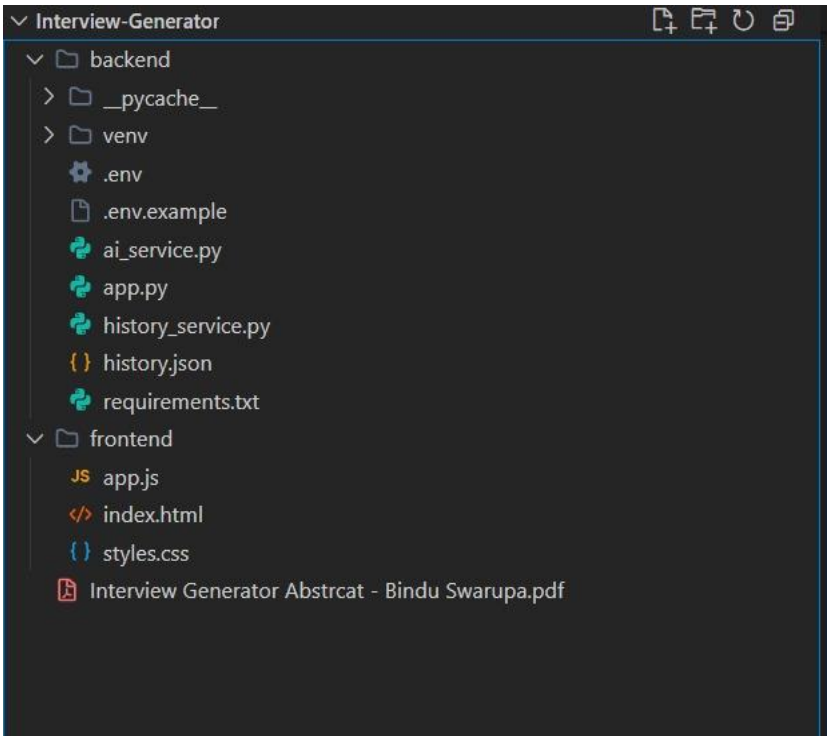


Figure: Project directory structure showing backend and frontend file organization.

Milestone 2: Core Functionalities Development

Milestone 2 focuses on building the core intelligence and data management modules that power the application. This includes the AI prompt engineering, persistent history storage, and professional document export capabilities.

Activity 2.1: Develop Core AI Question Generation Features

The core functionality of the AI system lies in constructing a robust, context-sensitive prompt that instructs Google Gemini to act as a world-class HR consultant and senior technical lead. We implemented this in ai_service.py using dynamic variables mapped from the user's frontend input. The prompt enforces a strict JSON schema to ensure reliable frontend parsing, and distributes questions based on the selected interview type:

- **Technical Mode:** Distributes questions heavily in favor of domain knowledge, system design, and coding practices (approx. 70% technical, 20% behavioral, 10% situational).
- **Behavioral Mode:** Focuses on soft skills, teamwork, and cultural alignment (approx. 60% behavioral, 30% situational, 10% technical).
- **Mixed Mode:** Provides a comprehensive, well-rounded assessment suitable for standard interviews (approx. 40% technical, 35% behavioral, 25% situational).

Activity 2.2: Develop Session History Management

InterviewAI includes a persistent history logging service (history_service.py) that writes session data locally to a structured JSON file (history.json). This ensures history is preserved across server restarts. Each history entry is logged with a unique UUID, a timestamp, and metadata containing the role, experience level, and requested question count, along with the raw AI question payload. This layout facilitates fast loading times without needing additional API requests.

Activity 2.3: Develop Word Document Export Service

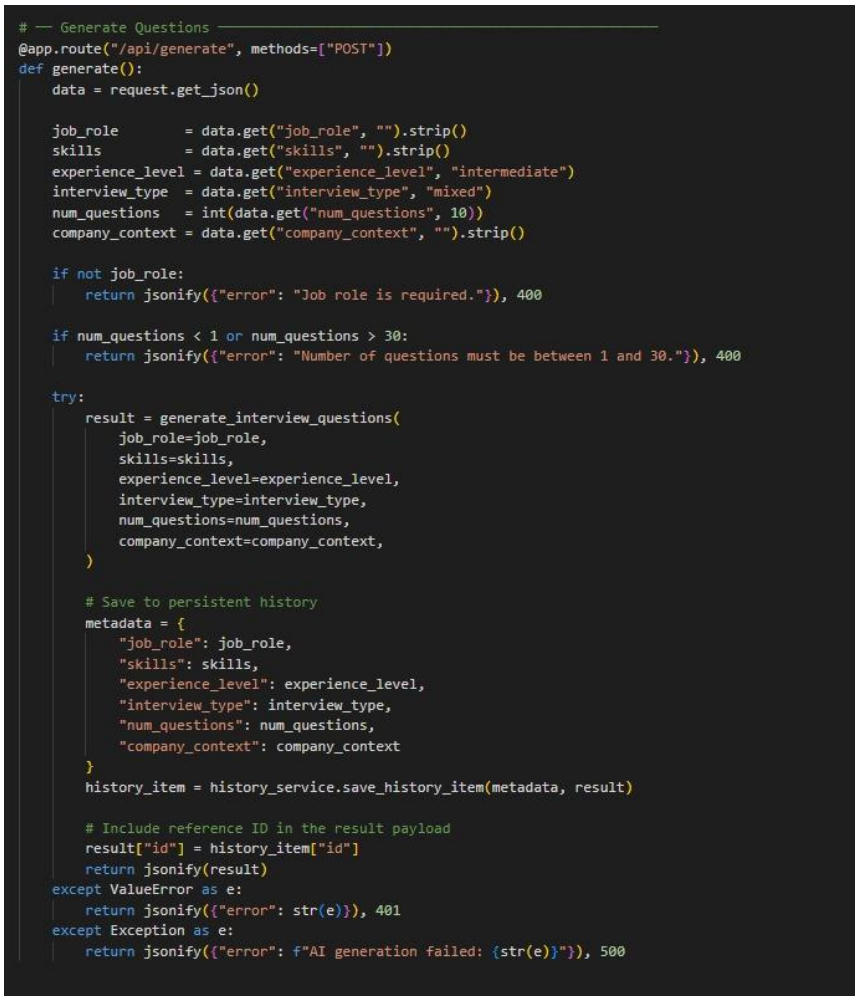
Using python-docx, we developed a high-quality, professional export service that compiles the generated questions, follow-ups, evaluation tips, and full scoring guides directly into a Microsoft Word document. The compiled file is built dynamically in the system's temporary directory, ensuring server disk space is managed efficiently, before being streamed securely to the user's web browser.

Milestone 3: App.py Backend Development

Milestone 3 focuses on implementing the Flask micro-framework backend, setting up CORS permissions, validating request data, and linking backend routes to our core AI and history services. Below are the key code implementations with visual references:

Activity 3.1: Generate Questions Endpoint (/api/generate)

The /api/generate endpoint accepts asynchronous JSON POST requests from the client. It validates input properties such as job role (required), question count range (1-30), and optional fields like skills and company context. Upon successful validation, it forwards the parameters to the Google Gemini service wrapper and saves the result to persistent history before returning the response.



```python
# ── Generate Questions ──────────────────────────────────────────
@app.route("/api/generate", methods=["POST"])
def generate():
    data = request.get_json()

    job_role        = data.get("job_role", "").strip()
    skills          = data.get("skills", "").strip()
    experience_level = data.get("experience_level", "intermediate")
    interview_type  = data.get("interview_type", "mixed")
    num_questions   = int(data.get("num_questions", 10))
    company_context = data.get("company_context", "").strip()

    if not job_role:
        return jsonify({"error": "Job role is required."}), 400

    if num_questions < 1 or num_questions > 30:
        return jsonify({"error": "Number of questions must be between 1 and 30."}), 400

    try:
        result = generate_interview_questions(
            job_role=job_role,
            skills=skills,
            experience_level=experience_level,
            interview_type=interview_type,
            num_questions=num_questions,
            company_context=company_context,
        )

        # Save to persistent history
        metadata = {
            "job_role": job_role,
            "skills": skills,
            "experience_level": experience_level,
            "interview_type": interview_type,
            "num_questions": num_questions,
            "company_context": company_context
        }
        history_item = history_service.save_history_item(metadata, result)

        # Include reference ID in the result payload
        result["id"] = history_item["id"]
        return jsonify(result)
    except ValueError as e:
        return jsonify({"error": str(e)}), 401
    except Exception as e:
        return jsonify({"error": f"AI generation failed: {str(e)}"}), 500
```

Figure: Generate Questions — The /api/generate POST route implementation showing input validation, AI service call, and history persistence.

Activity 3.2: Evaluation Criteria Structure

Each generated question set includes a comprehensive evaluation criteria section containing a 1-5 scoring scale, key competencies to assess, green flags indicating positive candidate signals, and red flags highlighting warning signs. This structured framework ensures recruiters conduct objective assessments.

```
# ── Evaluation Criteria ──────────────────────────────────────
ec = questions_data.get("evaluation_criteria", {})
if ec:
    doc.add_heading("Evaluation Criteria", level=1)

    if ec.get("scoring_scale"):
        p = doc.add_paragraph()
        p.add_run("Scoring Scale: ").bold = True
        p.add_run(ec["scoring_scale"])

    def add_list(heading, items, bullet="•"):
        if items:
            doc.add_heading(heading, level=2)
            for item in items:
                doc.add_paragraph(f"{bullet} {item}")

    add_list("Key Competencies", ec.get("key_competencies", []))
    add_list("Green Flags ✓", ec.get("green_flags", []), "✓")
    add_list("Red Flags ✗", ec.get("red_flags", []), "✗")

# ── Interview Tips ──────────────────────────────────────────
tips = questions_data.get("interview_tips", [])
if tips:
    doc.add_heading("Interview Tips", level=1)
    for tip in tips:
        doc.add_paragraph(f"• {tip}")

tmp = tempfile.NamedTemporaryFile(delete=False, suffix=".docx")
doc.save(tmp.name)
return tmp.name
```

Figure: Evaluation Criteria — Structured scoring scale, key competencies, green flags, and red flags builder in ai_service.py.

Activity 3.3: History Management Endpoints (/api/history)

To support history state synchronization between the client dashboard and the persistent database, we implemented RESTful CRUD endpoints in app.py. The GET /api/history endpoint returns metadata for all sessions, GET /api/history/<item_id> retrieves a specific session with full question data, and DELETE /api/history/<item_id> removes a session permanently from the JSON store.

```python
# ── History Management ──────────────────────────────────────────
@app.route("/api/history", methods=["GET"])
def get_history():
    try:
        return jsonify(history_service.get_history_list())
    except Exception as e:
        return jsonify({"error": f"Failed to retrieve history: {str(e)}"}), 500


@app.route("/api/history/<item_id>", methods=["GET"])
def get_history_item(item_id):
    try:
        item = history_service.get_history_item(item_id)
        if not item:
            return jsonify({"error": "History record not found"}), 404
        return jsonify(item)
    except Exception as e:
        return jsonify({"error": f"Failed to retrieve history item: {str(e)}"}), 500


@app.route("/api/history/<item_id>", methods=["DELETE"])
def delete_history_item(item_id):
    try:
        success = history_service.delete_history_item(item_id)
        if not success:
            return jsonify({"error": "History record not found"}), 404
        return jsonify({"status": "success", "message": "History item deleted"})
    except Exception as e:
        return jsonify({"error": f"Failed to delete history item: {str(e)}"}), 500


# ── Run ─────────────────────────────────────────────────────────
if __name__ == "__main__":
    port = int(os.getenv("PORT", 5000))
    print(f"\n[OK] Interview Generator API running at http://localhost:{port}\n")
    app.run(debug=True, port=port)
```

Figure: History Management — RESTful CRUD endpoints for session listing, retrieval, and deletion.

Activity 3.4: Export as DOCX & PDF (/api/export)

The /api/export endpoint accepts the active question set payload from the client browser and compiles it into a formatted Word (.docx) document using python-docx. The document includes structured sections for technical, behavioral, and situational questions, each with difficulty badges, follow-up prompts, and evaluation tips. The file is streamed back as a downloadable attachment.

```python
# ── Export as DOCX ──────────────────────────────────────────────
@app.route("/api/export", methods=["POST"])
def export():
    data = request.get_json()
    questions_data = data.get("questions_data", {})
    job_role       = data.get("job_role", "Interview")

    if not questions_data:
        return jsonify({"error": "No questions data provided."}), 400

    try:
        file_path = generate_docx(questions_data, job_role)
        safe_name = job_role.replace(" ", "_").replace("/", "-")
        return send_file(
            file_path,
            as_attachment=True,
            download_name=f"{safe_name}_Interview_Questions.docx",
            mimetype="application/vnd.openxmlformats-officedocument.wordprocessin
        )
    except Exception as e:
        return jsonify({"error": f"Export failed: {str(e)}"}), 500
```

Figure: Export Service — python-docx document compiler with custom formatting and file streaming.

Milestone 4: Frontend Development

Activity 4.1: Designing the User Interface (HTML & CSS)

The frontend is built as a pure Single-Page Application (SPA) using vanilla HTML5, CSS3, and ES6 JavaScript, eliminating heavy compilation frameworks. The core page contains three dedicated content sections structured inside container divs. Navigation and view toggles are managed purely client-side by applying or removing a helper .hidden utility class:

- **Page 1: Home Landing (#page-home):** Presents a premium hero overview, active stats indicator, a top right 'Let's Start' action, an 'Explore More' button, and a visual history summary of recent sessions.
- **Page 2: Configure Form (#page-form):** Displays a gorgeous form card with fields for job role, required skills, experience level selection, question count sliders, interview type selectors, and optional company context.
- **Page 3: Results Dashboard (#page-results):** Features a two-column interactive results space containing a history navigation sidebar on the left and a main results display tab panel on the right.

Activity 4.2: CSS Styling and Glassmorphism Aesthetics

To deliver a premium visual experience, the interface utilizes a dark-mode glassmorphism design. All styles are defined in styles.css using modern CSS custom properties for uniform token management:

- **Animated Glow Orbs:** Three blurred gradient background orbs (.orb-1, .orb-2, .orb-3) utilize CSS @keyframes animations to shift position, scale, and opacity slowly. This adds visual depth without impacting CPU usage.
- **Glassmorphism Cards:** Cards are styled with a semi-transparent background (rgba(255, 255, 255, 0.04)), custom borders, and a backing backdrop-filter: blur(20px) style. This allows the background glows to show through subtly.
- **Hover Transitions:** Micro-transitions are applied to all cards and buttons. Hover actions trigger scale changes (transform: translateY(-2px)), glow highlights, and icon transitions, creating a tactile feel.

Activity 4.3: Dynamic Client Logic in app.js

Client interaction is driven by app.js. It manages local application state, handles navigation transitions, and compiles Fetch requests to the server:

- **SPA Router Toggles:** The showPage(pageId) function controls the visible screen by dynamically adding the .hidden class to unused views while resetting the viewport scroll to the top of the window.
- **Dynamic Card Compilers:** Upon receiving JSON payloads from the server, template literal templates compile individual question elements recursively. This builds cards containing badge classes based on category and difficulty.
- **Evaluation Renderer:** A dedicated function parses green flags, red flags, competencies, and interviewer tips, rendering them into a tab panel that HR professionals can review side-by-side.
- **Sidebar History Toggles:** When history lists update, the sidebar and home grid synchronize, showing relative timestamps and permitting instant deletion or loading of past entries.

Milestone 5: Deployment & Verification

Activity 5.1: Preparing the Application for Local Deployment

Milestone 5 focuses on verifying, validating, and deploying the completed InterviewAI system. First, we perform local validation to ensure all API components communicate flawlessly.

- **Start the Flask App:** Activate the Python environment and launch the backend: python app.py. This serves the server on port 5000.
- **Browse Local Host:** Open a web browser and visit http://localhost:5000 to access the application directly through the local server.
- **Verify Health API:** Confirm connection by checking /api/health. This ensures that CORS headers are active and database links are online.
- **Validate Generation Flows:** Test all three types (Technical, Behavioral, Mixed) across multiple experience levels to confirm that output schemas are valid and error banners are handled correctly.

Below is the terminal output showing the Flask server starting successfully:

```
PS C:\Users\HP\Desktop\Interview-Generator\backend> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\HP\Desktop\Interview-Generator\backend> python app.py

[OK] Interview Generator API running at http://localhost:5000

 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 123-456-789
```

Activity 5.3: Public Deployment via Ngrok

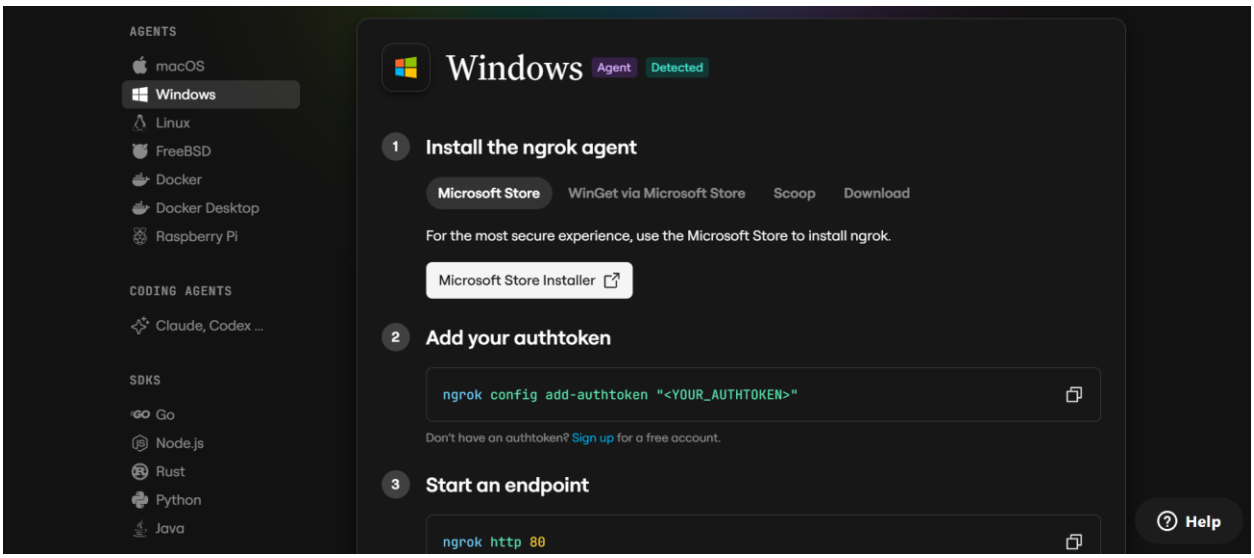
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

  pip install pyngrok

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on “Microsoft Store Installer”.



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:
  ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:
  python run_public.py
- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:
  ==========
  YOUR PUBLIC WEBSITE URL IS LIVE!
  URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
  ==========
- Share this URL with anyone who can access your Interview AI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

Exploring the Website's Web Pages

Home / Landing Page:

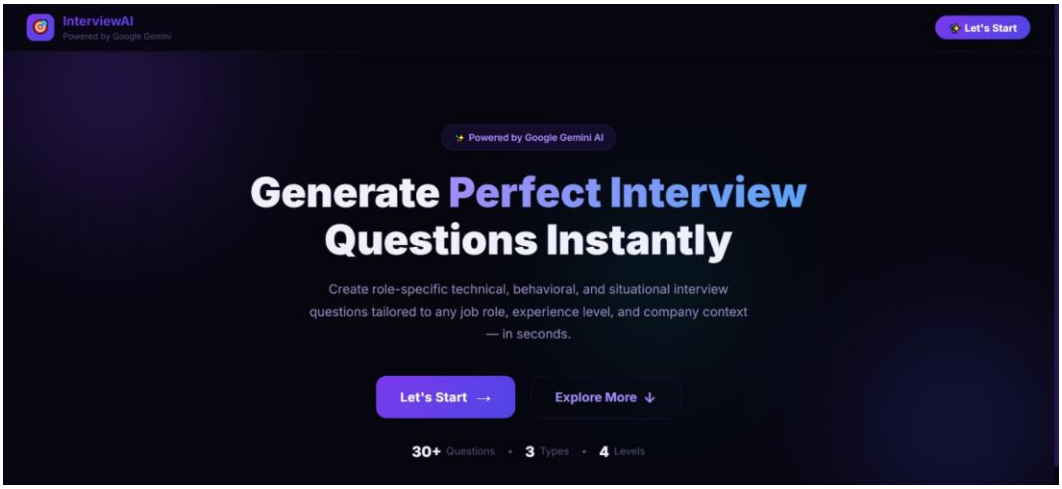


Figure: Main Landing Page — Premium centered hero view with CTA buttons and live statistics.

Description: The landing page presents a premium, centered hero section with the application title, a brief tagline, and two call-to-action buttons — 'Let's Start' to navigate directly to the interview configuration form, and 'Explore More' to reveal additional information sections below. The header displays the InterviewAI branding with a 'Powered by Google Gemini' badge, and live statistics show 30+ questions, 3 types, and 4 levels.

Simple Process Section:

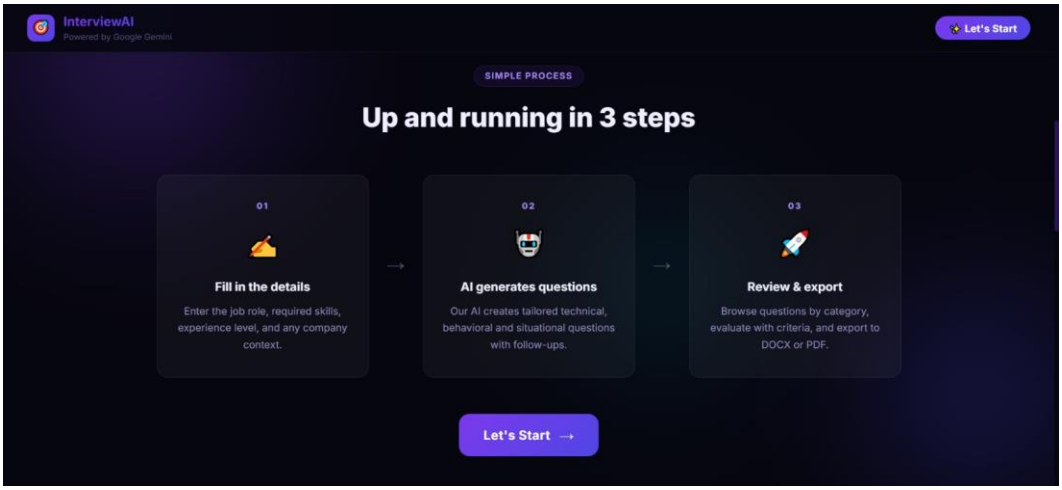


Figure: Simple Process Section — Three-step workflow cards with animated reveal transition.

Description: When the user clicks the 'Explore More' button, the lower information panels slide into view seamlessly. The 'Simple Process' section showcases a three-step workflow: (1) Fill in the details — enter job role, skills, and experience level; (2) AI generates questions — the system creates tailored technical, behavioral, and situational questions with follow-ups; (3) Review & export — browse questions by category, evaluate with criteria, and export to DOCX or PDF.

What You Get Section:

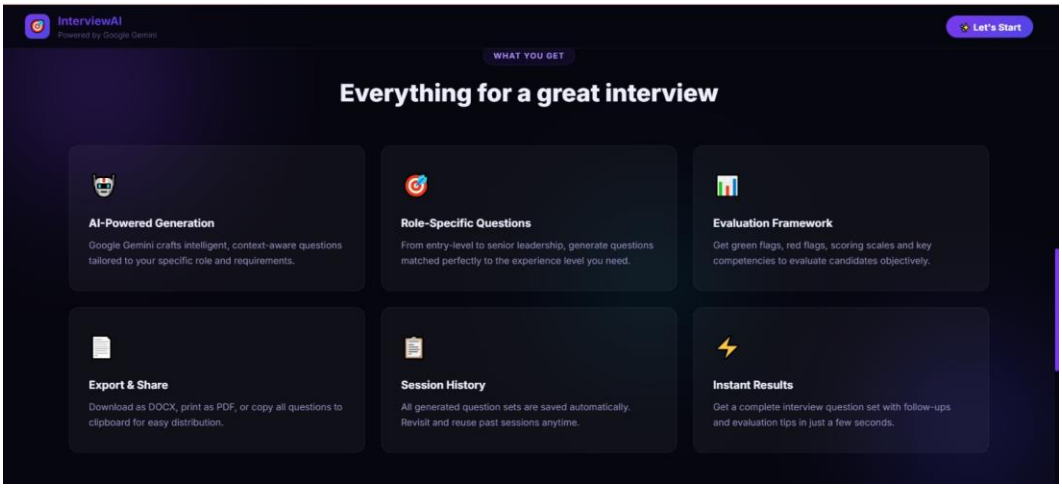


Figure: What You Get Section — Six-card feature grid displaying platform capabilities.

Description: The 'What You Get' section presents a six-card feature grid highlighting the platform's core capabilities: AI-Powered Generation (Google Gemini crafts intelligent questions), Role-Specific Questions (matched to experience levels), Evaluation Framework (green flags, red flags, scoring scales), Export & Share (DOCX, PDF, clipboard), Session History (automatic saving and retrieval), and Instant Results (complete question sets generated in seconds).

Configure Interview Form:

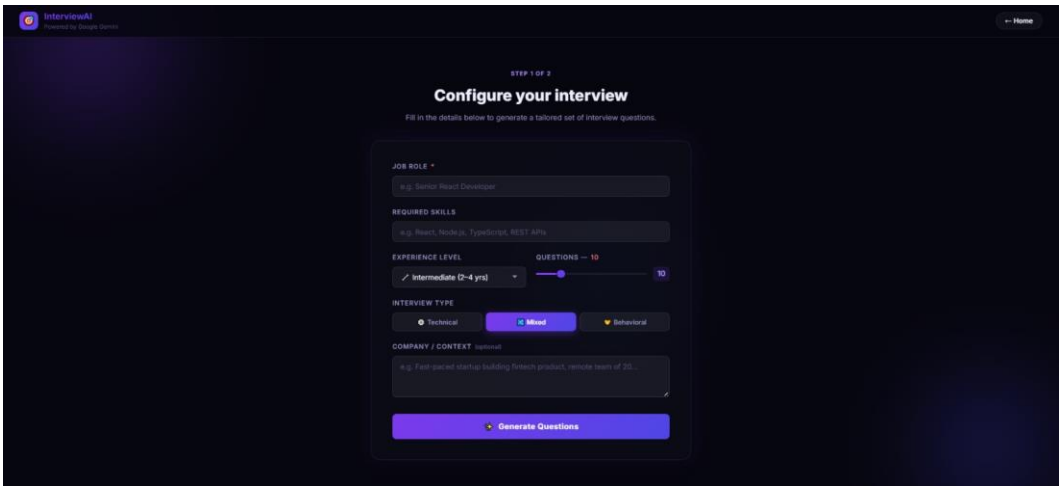


Fig: Configure Interview Form — Responsive form card with sliders, dropdowns, and toggle buttons.

Description: The configuration form page (Step 1 of 2) provides an intuitive card layout with the following input fields: Job Role (required text input), Required Skills (optional comma-separated list), Experience Level (dropdown selector with Beginner, Intermediate, Advanced, and Senior options), Questions count (interactive slider ranging from 5 to 30), Interview Type (toggle buttons for Technical, Mixed, and Behavioral), and Company Context (optional textarea for organizational details). The 'Generate Questions' button triggers the asynchronous API call with a loading spinner.

Results Dashboard:

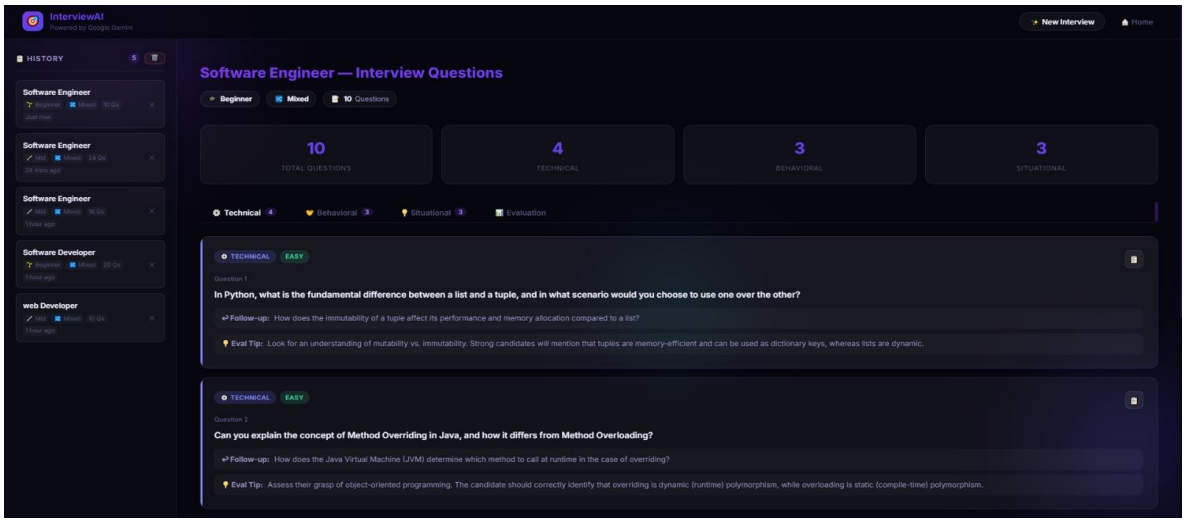


Figure: Results Dashboard — Two-column layout with history sidebar, stats cards, and question cards.

Description: The results dashboard features a two-column layout with a history sidebar on the left and the main results panel on the right. The top section displays statistics cards showing total questions, technical count, behavioral count, and situational count. Below, a tabbed navigation allows switching between Technical, Behavioral, Situational, and Evaluation views. Each question card displays the category badge, difficulty level, the question text, a follow-up prompt, and an evaluation tip.

Evaluation Panel:

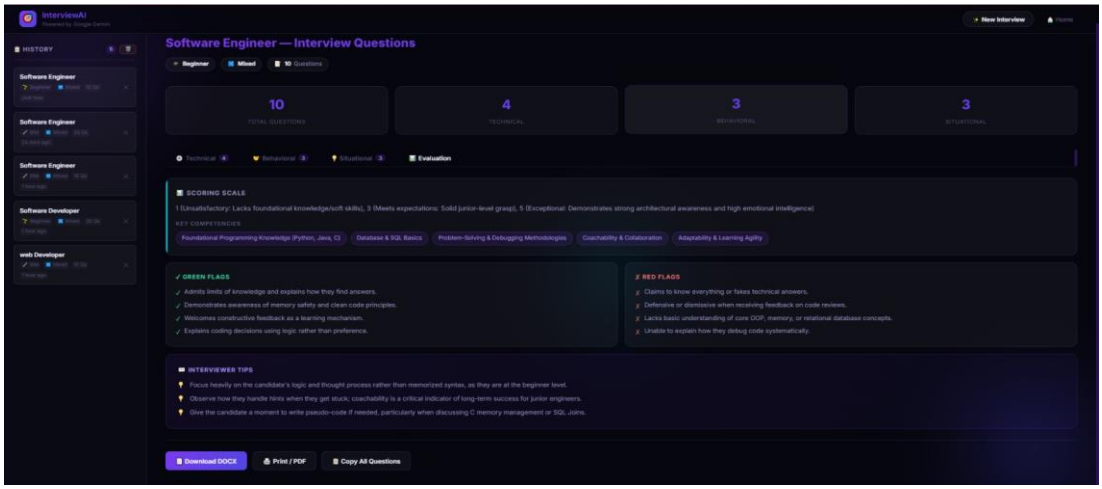


Figure: Evaluation Panel — Scoring scale, competencies, green flags, red flags, and interviewer tips.

Description: The Evaluation tab presents a structured assessment framework containing: Scoring Scale (1-5 rating from Unsatisfactory to Exceptional), Key Competencies (tagged pills showing skills to assess), Green Flags (positive candidate signals like 'Admits limits of knowledge', 'Demonstrates awareness of memory safety'), Red Flags (warning signs like 'Claims to know everything', 'Defensive when receiving feedback'), and Interviewer Tips (actionable guidance for conducting the interview effectively).

Print / Save as PDF:

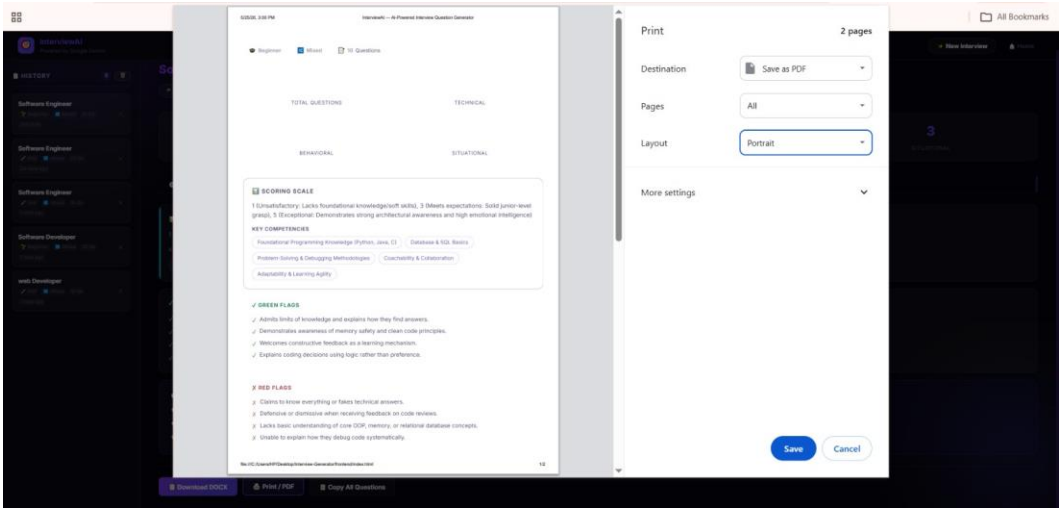


Figure: Print / Save as PDF — Browser print dialog showing clean, optimized print layout.

Description: The application supports browser-native printing via the 'Print / PDF' button. The print dialog displays a clean, optimized layout with proper margins, showing the evaluation criteria, key competencies, green flags, and red flags in a printer-friendly format. Users can save directly as PDF using their browser's built-in 'Save as PDF' destination option.

Conclusion

InterviewAI is a highly professional, modern, and standard-compliant recruiter empowerment system. By integrating the speed of Google Gemini API with a sleek glassmorphism client and robust local JSON session tracking, it sets a premium standard for hiring tools. The system eliminates repetitive administrative preparation work, allowing HR professionals and technical managers to focus entirely on candidate engagement and quality assessment.

The platform demonstrates end-to-end full-stack development proficiency, covering secure API integration, RESTful backend design, modern CSS aesthetics, asynchronous JavaScript programming, and professional document compilation. It establishes a strong foundation for future enterprise features, including multi-user collaborative reviews, live audio candidate screening, and cloud-hosted deployment.

Key achievements of this project include: successful integration with Google Gemini API for instant AI-powered question generation, a complete evaluation framework with scoring scales and behavioral flags, persistent session history with CRUD management, professional Word document export capabilities, a premium dark-mode glassmorphism user interface, and responsive design optimized for desktop and mobile devices.